

MCDI505 — Análisis de Series de Tiempo

Sumativa 1 · Informe de análisis exploratorio de series de tiempo

Consumo eléctrico mensual del segmento comercial — Distribuidora Eléctrica del Litoral (DEL)

Autor: Roberto Moncada

Programa: Magíster en Ciencia de Datos · UNAB Online

Curso: MCDI505 — Análisis de Series de Tiempo · Unidad 1, Semana 1

Archivo: mcdi505_slrmoncada.ipynb · **Fecha:** septiembre de 2026

Resumen. Este informe estudia el consumo eléctrico mensual del segmento comercial de DEL entre 2015 y 2023 (108 meses de entrenamiento) con gráficos, descomposición clásica (tendencia, estacionalidad y error) y funciones de autocorrelación. La tendencia crece hasta 2018 y luego se estanca; la estacionalidad anual es muy fuerte (fuerza $F_s = 0,90$, en una escala de 0 a 1), con máximo en enero y mínimo en mayo; el COVID-19 produjo una caída entre marzo de 2020 y enero de 2022 equivalente a unos 195 GWh no consumidos entre marzo de 2020 y diciembre de 2021; hay un dato atípico en noviembre de 2017 y un posible cambio de nivel en abril de 2019; y el residuo conserva autocorrelación ($\rho_1 = 0,44$). Como la ACF decae lentamente y tiene picos en los rezagos 12, 24, 36 y 48, la serie **no es estacionaria** y es estacional. Para el modelamiento propongo una transformación logarítmica, una diferenciación estacional ($D=1$), probablemente una regular ($d=1$), y un tratamiento explícito del quiebre por COVID.

Palabras clave: series de tiempo, estacionalidad, descomposición, ACF, PACF, consumo eléctrico.

Organización. El informe sigue el orden de los criterios de evaluación: (1) contexto, (2) preparación de datos, (3) exploración, (4) descomposición, (5) autocorrelación, (6) interpretación integrada, (7) conclusiones y (8) referencias. Cada decisión metodológica se presenta con cuatro partes: qué decidí, la evidencia, la alternativa descartada y las consecuencias.

Conceptos clave. *Tendencia:* dirección de largo plazo. *Estacionalidad:* patrón que se repite cada año ($s=12$ meses). *Ciclo:* subidas y bajadas de largo plazo sin periodo fijo. *Residuo:* lo que queda al quitar tendencia y estacionalidad. *Estacionariedad:* promedio y variabilidad que no cambian con el tiempo. *ACF:* correlación de la serie consigo misma desplazada k meses (rezago). *PACF:* lo mismo, descontando los rezagos intermedios.

1. Introducción y contexto del dataset

Distribuidora Eléctrica del Litoral (DEL) distribuye electricidad en comunas costeras y del interior de la Región de Valparaíso. Su segmento comercial (oficinas, comercio, hotelería y servicios) representa cerca de un tercio de la energía distribuida y es el que más varía según la época del año. Cada diciembre, DEL informa al Coordinador Eléctrico Nacional cuánta energía contratará mes a mes el año siguiente. El error es asimétrico: si calcula de menos, compra el faltante en el mercado *spot* a dos a cuatro veces el precio contratado; si calcula de más, paga energía que no se consume ni se revende.

La pregunta de negocio es cuánto consumo mensual se espera para los próximos doce meses, con qué incertidumbre y qué volumen contratar. Este informe es la **primera etapa**: entender cómo está construida la serie (tendencia, estacionalidad, ciclo y error) y relacionarla con la ACF y la PACF. No se construye ningún modelo de pronóstico.

Datos. El archivo `consumo_electrico_litoral.csv` tiene 120 observaciones mensuales (enero de 2015 a diciembre de 2024) de cuatro variables: consumo facturado (`consumo_gwh`, en GWh; es la variable de estudio), empalmes comerciales (`clientes_comerciales`), temperatura media (`temperatura_media_c`) y días hábiles (`dias_habiles`). Los datos son sintéticos y reproducen la estructura de una serie real (Universidad Andrés Bello, s. f.). El periodo estacional es $s=12$.

Partición. Siguiendo el protocolo del curso, uso solo **2015-01 a 2023-12 (108 observaciones)**. El año 2024 es el conjunto de prueba, queda sellado y no interviene en ningún cálculo: mirar el futuro al tomar decisiones contaminaría la evaluación final (Tashman, 2000).

Hechos del periodo (contrastados con los datos): ingreso de unos 900 clientes por un centro comercial en abril de 2019, restricciones por COVID-19 entre marzo de 2020 y diciembre de 2021, y una recalibración de medidores en noviembre de 2017 que dejó un registro anómalo. Las otras variables se usan solo para explorar (sección 3.5); el enfoque es univariado.

2. Configuración y preparación de los datos

Fijo una semilla aleatoria, uso rutas relativas y no modifico el archivo original, de modo que el notebook sea reproducible. Luego verifico la integridad de la serie. Uso `pandas`, `matplotlib` y `statsmodels` (Seabold y Perktold, 2010); el código está comentado.

▼ Código

```
# =====
# 2.1 Librerías y configuración general
# =====
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import acf, pacf
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

warnings.filterwarnings("ignore")
np.random.seed(505) # semilla fija (reproducibilidad)

# Paleta sobria y consistente en todo el informe
ROJO, AZUL, VERDE, GRIS, NEGRO = "#b30338", "#0057b8", "#2e7d32", "#9e9e9e", "#222222"
plt.rcParams.update({
    "figure.figsize": (11, 3.8), "figure.dpi": 100,
    "axes.grid": True, "grid.alpha": 0.25,
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.titlesize": 11, "axes.labelsize": 9.5, "font.size": 9,
})

# Hitos del caso (contexto entregado en el documento base del curso)
COVID_INI, COVID_FIN = pd.Timestamp("2020-03-01"), pd.Timestamp("2021-12-01")
CLIENTES = pd.Timestamp("2019-04-01") # incorporación de ~900 clientes comerciales
MEDIDORES = pd.Timestamp("2017-11-01") # recalibración de medidores (atípico)
print("pandas", pd.__version__, "· numpy", np.__version__)
```

pandas 3.0.2 · numpy 2.4.4

▼ Código

```
# =====
# 2.2 Carga de datos (ruta relativa) y SELLADO del conjunto de prueba
# =====
ARCHIVO = "consumo_electrico_litoral.csv"
ruta = next(p for p in [Path(ARCHIVO), Path("datos") / ARCHIVO, Path("../") / "datos" / ARCHIVO]
             if p.exists())

# asfreq("MS") fuerza el calendario mensual: un mes ausente quedaría como NaN
df_total = pd.read_csv(ruta, parse_dates=["fecha"]).set_index("fecha").asfreq("MS")

# Partición cronológica: 2015-01 a 2023-12 (entrenamiento) | 2024 (prueba sellada).
# A partir de esta línea NO se vuelve a usar el año 2024 en ningún cálculo ni gráfico.
CORTE = "2023-12-31"
df = df_total.loc[:CORTE].copy()
del df_total

print(f"Archivo: {ruta}")
print(f"Periodo analizado: {df.index.min():%Y-%m} a {df.index.max():%Y-%m} "
      f"({len(df)} observaciones = {len(df)//12} años completos)")
display(df.head())
```

Archivo: consumo_electrico_litoral.csv
Periodo analizado: 2015-01 a 2023-12 (108 observaciones = 9 años completos)

	consumo_gwh	clientes_comerciales	temperatura_media_c	dias_habiles
fecha				
2015-01-01	108.32	41827	19.6	22
2015-02-01	94.71	41918	18.7	20
2015-03-01	89.52	42045	16.5	22
2015-04-01	82.24	42129	13.3	22
2015-05-01	82.60	42242	12.0	21

▼ Código

```
# =====
# 2.3 Ficha técnica y auditoría de integridad
# =====
print("Frecuencia del índice :", df.index.freqstr)
print("Índice ordenado      :", df.index.is_monotonic_increasing)
print("Índice sin duplicados :", df.index.is_unique)
completo = pd.date_range(df.index.min(), df.index.max(), freq="MS")
print("Calendario completo  :", df.index.equals(completo))
```

```
print("Consumo <= 0      :", int((df["consumo_gwh"] <= 0).sum()))
print("Nulos en consumo  :", df["consumo_gwh"].isna().sum(),
      list(df.index[df["consumo_gwh"].isna()].strftime("%Y-%m")))

display(df.describe().T.round(2))
```

```
Frecuencia del índice : MS
Índice ordenado       : True
Índice sin duplicados  : True
Calendario completo   : True
Consumo <= 0          : 0
Nulos en consumo      : 3 ['2016-08', '2019-02', '2022-06']
```

	count	mean	std	min	25%	50%	75%	max
consumo_gwh	105.0	102.85	12.67	73.19	94.71	101.31	111.08	133.58
clientes_comerciales	108.0	48634.68	4191.69	41827.00	44891.75	48952.50	52340.75	55436.00
temperatura_media_c	108.0	15.59	3.20	10.50	12.50	15.30	18.40	21.70
dias_habiles	108.0	21.73	0.94	20.00	21.00	22.00	22.00	23.00

Lectura de la auditoría. El índice está ordenado, sin duplicados y coincide con el calendario mensual completo, por lo que los datos están igualmente espaciados. No hay consumos menores o iguales a cero. El único problema son **tres valores faltantes** (agosto de 2016, febrero de 2019 y junio de 2022), un 2,8 % de las 108 observaciones, atribuidos a fallas de telemetría. El consumo promedia 102,85 GWh (desviación estándar 12,67 GWh), con mínimo de 73,19 y máximo de 133,58 GWh.

▼ Código

```
# =====
# 2.4 Valores faltantes: comparación de estrategias y decisión
# =====
s = df["consumo_gwh"]
huecos = s[s.isna()].index

estrategias = {
    "lineal": s.interpolate("linear"),
    "temporal": s.interpolate("time"),
    # mismo mes del año previo, escalado por la razón entre el mes anterior y su par del año previo
    "estacional": s.fillna(s.shift(12) * (s.shift(1) / s.shift(13))),
}
comp = pd.DataFrame({k: v.loc[huecos] for k, v in estrategias.items()}).round(2)
comp.insert(0, "mes", huecos.month)
comp["máx-mín"] = (comp[["lineal", "temporal", "estacional"]].max(axis=1)
                  - comp[["lineal", "temporal", "estacional"]].min(axis=1)).round(2)
display(comp)

# Decisión: interpolación temporal para los tres huecos (ver justificación en el texto)
y = s.interpolate("time")
y.name = "consumo_gwh"

# Impacto de la decisión sobre media, desviación estándar y autocorrelación de orden 1
impacto = pd.DataFrame({
    k: {"media": v.mean(), "desv. estándar": v.std(), "ACF(1)": v.autocorr(1)}
    for k, v in {"lineal": estrategias["lineal"], "temporal": estrategias["temporal"]}.items()
}).T
display(impacto.round(4))
```

	mes	lineal	temporal	estacional	máx-mín
fecha					
2016-08-01	8	95.86	95.86	94.87	0.99
2019-02-01	2	116.93	116.34	114.18	2.75
2022-06-01	6	101.40	101.43	103.86	2.46

	media	desv. estándar	ACF(1)
lineal	102.9001	12.5828	0.7672
temporal	102.8950	12.5768	0.7670

Decisión 1 — Valores faltantes

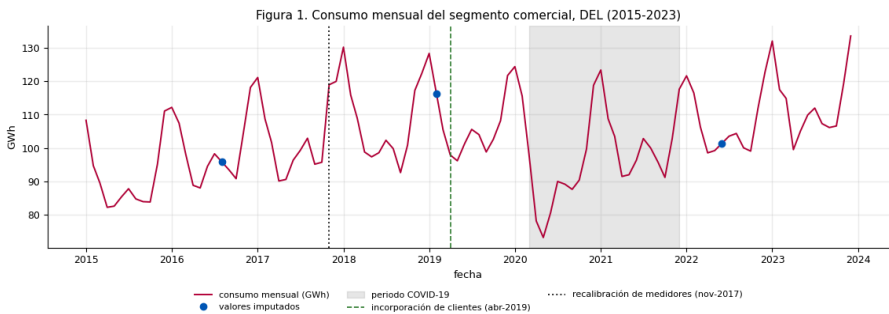
Elemento	Detalle
Qué decidí	Rellenar los tres huecos con interpolación temporal (<code>interpolate("time")</code>), porque ACF y descomposición no admiten huecos.
Evidencia	Los huecos caen en meses distintos (agosto en la zona plana de invierno, índice 0,96; febrero en el descenso desde enero, 1,21 a 1,01; junio cerca del valle de mayo, 0,89), pero las tres estrategias difieren solo 0,99, 2,75 y 2,46 GWh (a lo más 2,4 % del nivel) y en los tres casos los meses vecinos son datos reales.
Alternativa descartada	La imputación estacional depende de un único valor del año previo; además, para junio de 2022 el mes base (junio de 2021) estaba en recuperación del COVID (+19,5 % interanual) y habría traspasado esa distorsión.
Consecuencia	Efecto despreciable: media de 102,900 a 102,895 GWh, desviación de 12,583 a 12,577 y ACF(1) de 0,7672 a 0,7670. Declaro igual que 3 de 108 datos son imputados, porque rellenar vuelve artificialmente parecidos a los meses vecinos al hueco.

3. Exploración y visualización de la serie

3.1 Serie completa

▼ Código

```
# =====
# 3.1 Serie completa con hitos y valores imputados
# =====
fig, ax = plt.subplots(figsize=(11.5, 4.2))
ax.plot(y.index, y.values, color=R0J0, lw=1.4, label="consumo mensual (GWh)")
ax.plot(huecos, y.loc[huecos], "o", color=AZUL, ms=6, zorder=5, label="valores imputados")
ax.axvspan(COVID_INI, COVID_FIN, color=GRIS, alpha=0.25, label="periodo COVID-19")
ax.axvline(CLIENTES, color=VERDE, ls="--", lw=1.2, label="incorporación de clientes (abr-2019)")
ax.axvline(MEDIDORES, color=NEGRO, ls=":", lw=1.4, label="recalibración de medidores (nov-2017)")
ax.set_ylabel("GWh"); ax.set_xlabel("fecha")
ax.set_title("Figura 1. Consumo mensual del segmento comercial, DEL (2015-2023)")
ax.legend(fontsize=8, ncol=3, loc="upper center", bbox_to_anchor=(0.5, -0.16), frameon=False)
plt.tight_layout(); plt.show()
```



Descripción (Figura 1). La serie tiene un **patrón estacional anual** que se repite casi igual cada año: picos en verano (diciembre-enero), valles en otoño (abril-mayo) y un pico suave en invierno. Sobre él se distinguen cuatro rasgos:

- 1. **Tendencia creciente 2015-2018:** la media anual sube de 90,8 a 107,1 GWh.
- 2. **Estancamiento en 2018-2019** (107,1 y 107,2 GWh), aunque los clientes siguen creciendo.
- 3. **Caída abrupta desde marzo de 2020**, con mínimo histórico de 73,2 GWh en mayo de 2020 (se esperaba cerca de 97), recuperación gradual en 2021 y normalización en 2022. La media de 2020 (95,5 GWh) es 11 % menor que la de 2019.
- 4. **Máximo aislado** en noviembre de 2017 (sección siguiente).

La variabilidad alrededor del nivel parece estable, salvo en el COVID, cuando las oscilaciones aumentan por el hundimiento de otoño-invierno de 2020. No se ve un escalón en abril de 2019 (sección 3.5).

3.2 Variación interanual y dato atípico de noviembre de 2017

▼ Código

```
# =====
# 3.2 Variación interanual y aislamiento del atípico de 2017-11
# =====
yoy = (y / y.shift(12) - 1) * 100 # % respecto del mismo mes del año anterior

fig, axes = plt.subplots(1, 2, figsize=(12, 3.8), gridspec_kw={"width_ratios": [1.5, 1]})
axes[0].plot(yoy.index, yoy.values, color="#455a64", lw=1.2)
axes[0].axhline(0, color="k", lw=0.8)
axes[0].axvspan(COVID_INI, COVID_FIN, color=GRIS, alpha=0.25)
axes[0].annotate("2017-11", (MEDIDORES, yoy.loc[MEDIDORES]), xytext=(-55, 12),
                 textcoords="offset points", fontsize=8, arrowprops=dict(arrowstyle="-", lw=0.6))
axes[0].set_ylabel("% var. interanual")
axes[0].set_title("Figura 2a. Variación interanual del consumo")

# 2016, 2017 y 2018 superpuestos para aislar el dato de noviembre de 2017
for anio, col, lw in [(2016, GRIS, 1.2), (2017, ROJO, 1.8), (2018, AZUL, 1.2)]:
    g = y[anio]
    axes[1].plot(g.index.month, g.values, color=col, lw=lw, marker="o", ms=3, label=str(anio))
axes[1].set_xticks(range(1, 13)); axes[1].set_xlabel("mes"); axes[1].set_ylabel("GWh")
axes[1].set_title("Figura 2b. Años 2016-2018 superpuestos"); axes[1].legend(fontsize=8)
plt.tight_layout(); plt.show()

# z robusto del componente irregular (mismo procedimiento que la auditoría del curso):
# 1) nivel = media móvil centrada 2x12; 2) razón serie/nivel; 3) dividir por la mediana del mes
nivel = y.rolling(12, center=True).mean().rolling(2, center=True).mean().shift(-1)
razon = (y / nivel).dropna()
irregular = razon / razon.groupby(razon.index.month).transform("median")
mad = (irregular - irregular.median()).abs().median()
z = (irregular - irregular.median()) / (1.4826 * mad)
candidatos = z[z.abs() > 3.5].round(2)
print("Candidatos a atípico (|z robusto| > 3,5):")
print(candidatos.rename("z").to_string())
print(f"\nNoviembre de 2017: {y.loc[MEDIDORES]:.2f} GWh | octubre: {y.loc['2017-10-01']:.2f} | "
      f"diciembre: {y.loc['2017-12-01']:.2f} | nov-2016: {y.loc['2016-11-01']:.2f} | nov-2018: {y.loc['2018-11-01']:.2f}")

# Efecto de reemplazar el dato por su valor esperado (nivel x razón mediana de noviembre)
esperado = nivel.loc[MEDIDORES] * razon.groupby(razon.index.month).median().loc[11]
y_aj = y.copy(); y_aj.loc[MEDIDORES] = esperado
print(f"Valor esperado para nov-2017: {esperado:.2f} GWh (exceso observado: {y.loc[MEDIDORES] - esperado:+.2f} GWh)")
print(f"Efecto de ajustarlo -> media: {y.mean():.3f} vs {y_aj.mean():.3f} | desv. estándar: {y.std():.3f} vs {y_aj.std():.3f}")
print(f"Variación interanual de nov-2017: {yoy.loc[MEDIDORES]:+.1f}% | de nov-2018: {yoy.loc['2018-11-01']:+.1f}%")
```

Candidatos a atípico (|z robusto| > 3,5):

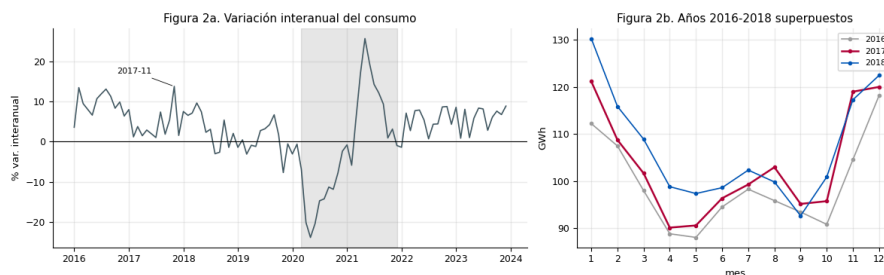
fecha	
2017-11-01	4.10
2019-10-01	3.56
2020-02-01	3.81
2020-04-01	-5.27
2020-05-01	-7.56
2020-06-01	-5.27

Noviembre de 2017: 118.96 GWh | octubre: 95.74 | diciembre: 119.97 | nov-2016: 104.56 | nov-2018: 117.25

Valor esperado para nov-2017: 109.86 GWh (exceso observado: +9.10 GWh)

Efecto de ajustarlo -> media: 102.895 vs 102.811 | desv. estándar: 12.577 vs 12.498

Variación interanual de nov-2017: +13.8% | de nov-2018: -1.4%



Descripción (Figuras 2a y 2b). La variación interanual compara cada mes con el mismo mes del año anterior y elimina la estacionalidad. Entre 2016 y 2019 promedia +4,4 % (de -7,7 % a +13,8 %): crecimiento moderado hasta 2018 y desaceleración después. Noviembre de 2017 marca +13,8 %, similar a febrero de 2016 (+13,5 %), de modo que lo anómalo es que **rompe el patrón estacional**: el salto de octubre (95,74 GWh) a noviembre (118,96 GWh) es de +24 %, casi el doble del habitual. Con el **z robusto** del componente irregular (atípico si supera 3,5 en valor absoluto) obtiene $z=+4,10$. Otros candidatos son octubre de 2019 ($z=3,56$) y febrero de 2020 ($z=3,81$), marginales y sin hecho que los explique, y abril a junio de 2020 ($z=-5,27$; -7,56; -5,27), que corresponden al COVID. La caída de 2020 llega a -23,9 % en mayo y el rebote de abril a junio de 2021 (+17 % a +26 %) es un **efecto base**: compara meses recuperados con meses hundidos y no indica crecimiento real.

Decisión 2 — Atípico de noviembre de 2017

Elemento	Detalle
Qué decidí	Conservar el dato en esta etapa exploratoria, marcarlo y reemplazarlo por su valor esperado antes de modelar.
Evidencia	Exceso de +9,10 GWh sobre el esperado (109,86 GWh), un 8,3 %. Se trata de una recalibración de medidores, no de consumo real. Corregirlo cambia la media de 102,895 a 102,811 y la desviación de 12,577 a 12,498 GWh.
Alternativa descartada	Eliminarlo sin registro rompería el índice mensual y la trazabilidad; dejarlo sin marcar metería ruido falso en la ACF y en la estimación.
Consecuencia	Aparecerá como uno de los mayores residuos positivos (+7,0 %). Para el modelamiento lo trataré como faltante (interpolación) o con una variable indicadora.

3.3 Patrón estacional y su relación con la temperatura

▼ Código

```
# =====
# 3.3 Patrón estacional: perfil promedio, superposición anual y temperatura
# =====
perfil = y.groupby(y.index.month).mean() / y.mean() # índice estacional simple
temp_mes = df["temperatura_media_c"].groupby(df.index.month).mean()

fig, axes = plt.subplots(1, 3, figsize=(13, 3.8))
axes[0].bar(perfil.index, perfil.values, color=R0J0, alpha=0.85)
axes[0].axhline(1, color="k", lw=0.9)
axes[0].set_xticks(range(1, 13)); axes[0].set_xlabel("mes"); axes[0].set_ylabel("índice (media = 1)")
axes[0].set_title("Figura 3a. Perfil estacional promedio")

for anio, g in y.groupby(y.index.year):
    especial = anio in (2020, 2023)
    axes[1].plot(g.index.month, g.values, lw=1.8 if especial else 0.9,
                 color={2020: AZUL, 2023: R0J0}.get(anio, "#cccccc"),
                 label=str(anio) if especial else None)
axes[1].set_xticks(range(1, 13)); axes[1].set_xlabel("mes"); axes[1].set_ylabel("GWh")
axes[1].set_title("Figura 3b. Superposición anual (2020 y 2023 destacados)")
axes[1].legend(fontsize=8)

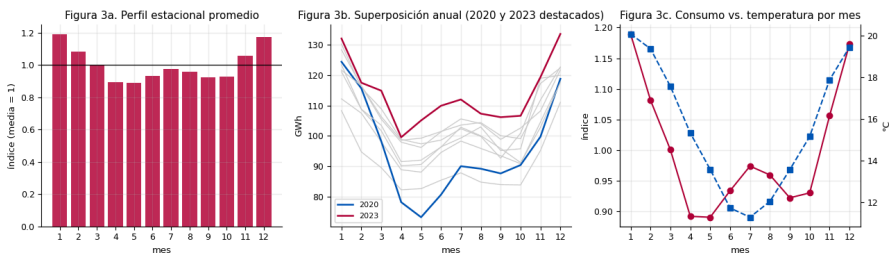
axes[2].plot(perfil.index, perfil.values, "o-", color=R0J0, label="índice estacional del consumo")
ax2 = axes[2].twinx()
ax2.plot(temp_mes.index, temp_mes.values, "s--", color=AZUL, label="temperatura media (°C)")
ax2.grid(False); ax2.spines["right"].set_visible(True)
axes[2].set_xticks(range(1, 13)); axes[2].set_xlabel("mes")
axes[2].set_ylabel("índice"); ax2.set_ylabel("°C")
axes[2].set_title("Figura 3c. Consumo vs. temperatura por mes")
plt.tight_layout(); plt.show()

print("Índice estacional por mes:", dict(zip(perfil.index, perfil.round(3))))
print("Temperatura media por mes:", dict(zip(temp_mes.index, temp_mes.round(1))))
print("Correlación mensual perfil-temperatura: "
      f"{np.corrcoef(perfil.values, temp_mes.values)[0, 1]:+.2f}")
```

Índice estacional por mes: {1: 1.19, 2: 1.081, 3: 1.0, 4: 0.892, 5: 0.89, 6: 0.933, 7: 0.974, 8: 0.959, 9: 0.922, 10: 0.93, 11: 1.056, 12: 1.173}

Temperatura media por mes: {1: 20.1, 2: 19.4, 3: 17.6, 4: 15.3, 5: 13.6, 6: 11.7, 7: 11.3, 8: 12.1, 9: 13.6, 10: 15.2, 11: 17.9, 12: 19.4}

Correlación mensual perfil-temperatura: +0.80



Descripción (Figuras 3a-3c). El perfil estacional tiene su **máximo en enero** (índice 1,19), niveles altos en diciembre (1,17) y febrero (1,08), **mínimo en abril-mayo** (0,89) y un repunte en **julio** (0,97) sobre junio (0,93) y agosto (0,96). El consumo de enero es cerca de un 34 % mayor que el de mayo. El perfil se repite cada año y solo 2020 lo rompe entre abril y octubre (Figura 3b).

Doble estacionalidad. La temperatura es máxima en enero (20,1 °C) y mínima en julio (11,3 °C). El consumo sube con el calor intenso (refrigeración y actividad turística, hipótesis coherente con el rubro hotelero) y, menos, con el frío (calefacción). Por tramos de temperatura promedia 97,1 GWh con 12 °C o menos, 95,0 y 95,3 entre 12 y 16 °C, 100,1 entre 16 y 18 °C y 116,8 sobre 18 °C: una **U asimétrica**, donde el frío eleva el consumo un 2 % sobre el mínimo y el calor un 23 %. Por eso domina el máximo de verano.

3.4 Estabilidad de la media y de la varianza

▼ Código

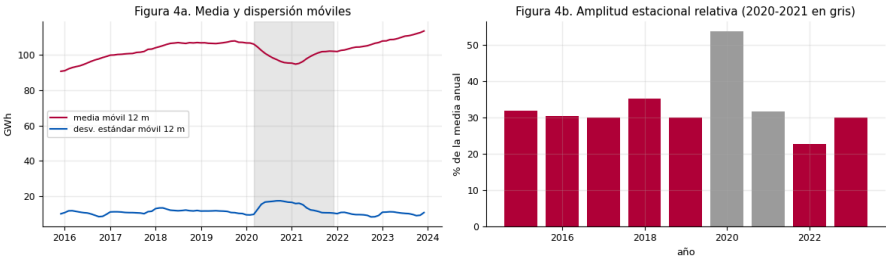
```
# =====
# 3.4 Nivel y dispersión: media/desv. móviles y amplitud estacional por año
# =====
fig, axes = plt.subplots(1, 2, figsize=(12, 3.6))
rm, rs = y.rolling(12).mean(), y.rolling(12).std()
axes[0].plot(rm, color=R0J0, label="media móvil 12 m")
axes[0].plot(rs, color=AZUL, label="desv. estándar móvil 12 m")
axes[0].axvspan(COVID_INI, COVID_FIN, color=GRIS, alpha=0.25)
axes[0].set_ylabel("GWh"); axes[0].legend(fontsize=8)
axes[0].set_title("Figura 4a. Media y dispersión móviles")

amp = pd.DataFrame({"media anual (GWh)": y.groupby(y.index.year).mean(),
                    "amplitud pico-valor (GWh)": y.groupby(y.index.year).max() - y.groupby(y.index.year).min()})
amp["amplitud relativa (%)"] = 100 * amp.iloc[:, 1] / amp.iloc[:, 0]
amp["CV intra-anual"] = y.groupby(y.index.year).std() / y.groupby(y.index.year).mean()
amp.index.name = "año"
axes[1].bar(amp.index, amp["amplitud relativa (%)"], color=[GRIS if a in (2020, 2021) else R0J0 for a in amp.index])
axes[1].set_ylabel("% de la media anual"); axes[1].set_xlabel("año")
axes[1].set_title("Figura 4b. Amplitud estacional relativa (2020-2021 en gris)")
plt.tight_layout(); plt.show()
display(amp.round(2))

normales = amp.drop(index=[2020, 2021])
print("Excluyendo 2020-2021: amplitud absoluta "
      f"{normales.iloc[0, 1]:.1f} -> {normales.iloc[-1, 1]:.1f} GWh; relativa "
      f"{normales.iloc[0, 2]:.1f}% -> {normales.iloc[-1, 2]:.1f}% "
      f"(desv. estándar de la relativa: {normales.iloc[:, 2].std():.2f} pp)")
# Elasticidad de la dispersión respecto del nivel (años sin COVID): 0 = aditivo puro, 1 = multiplicativo puro
lm = np.log(y.groupby(y.index.year).mean().drop(index=[2020, 2021]))
ls = np.log(y.groupby(y.index.year).std().drop(index=[2020, 2021]))
print(f"Elasticidad de la desv. estándar anual respecto de la media anual: {np.polyfit(lm, ls, 1)[0]:.2f}")
print("Correlación media móvil-desv. móvil antes de COVID (2015-2019): "
      f"{np.corrcoef(rm[:'2019-12'].dropna(), rs[:'2019-12'].dropna())[0, 1]:+.2f}")

Excluyendo 2020-2021: amplitud absoluta 28.8 -> 34.1 GWh; relativa 31.8% -> 30.0% (desv. estándar de la relativa: 3.76 pp)
Elasticidad de la desv. estándar anual respecto de la media anual: 0.31
Correlación media móvil-desv. móvil antes de COVID (2015-2019): +0.43
```

	media anual (GWh)	amplitud pico-valor (GWh)	amplitud relativa (%)	CV intra-anual
año				
2015	90.78	28.84	31.77	0.11
2016	99.18	30.11	30.36	0.10
2017	103.39	31.01	29.99	0.11
2018	107.09	37.59	35.10	0.11
2019	107.21	32.18	30.02	0.09
2020	95.49	51.21	53.63	0.17
2021	102.15	32.19	31.51	0.10
2022	107.11	24.16	22.56	0.08
2023	113.66	34.05	29.96	0.09



Descripción (Figuras 4a y 4b). La media móvil de 12 meses sube de 90,8 GWh (diciembre de 2015) a 106,9 (noviembre de 2018), cae en el COVID y llega a 113,7 a fines de 2023: **la media no es constante**, lo que descarta estacionariedad en media. La desviación móvil se mantiene entre 8 y 13 GWh fuera del shock (17 en 2020) y no sigue claramente al nivel (correlación +0,43 antes del COVID). La amplitud estacional relativa se mantiene cerca de 30 % (31,8 % en 2015, 30,0 % en 2023, máximo 35,1 % en 2018) mientras la absoluta crece de 28,8 a 34,1 GWh; en 2020 se dispara a 53,6 %.

Lectura. La evidencia es **mixta, con inclinación al esquema multiplicativo**: amplitud relativa estable y absoluta creciente indican efecto proporcional al nivel, pero la elasticidad de la dispersión anual respecto del nivel es solo 0,31 (1 sería multiplicativo puro, 0 aditivo puro). La decisión se toma en la sección 4.

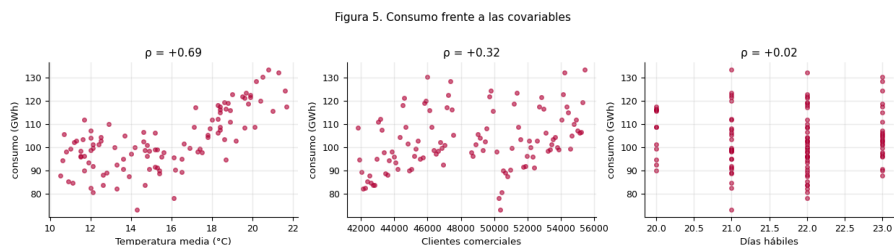
3.5 Relación con las covariables

▼ Código

```
# =====
# 3.5 Relación con las covariables (uso explicativo)
# =====
fig, axes = plt.subplots(1, 3, figsize=(12.5, 3.3))
for ax, var, etq in zip(axes, ["temperatura_media_c", "clientes_comerciales", "dias_habiles"],
                        ["Temperatura media (°C)", "Clientes comerciales", "Días hábiles"]):
    ax.scatter(df[var], y, s=16, alpha=0.6, color=R0J0)
    ax.set_xlabel(etq); ax.set_ylabel("consumo (GWh)")
    ax.set_title(f" $\rho = \{np.corrcoef(df[var], y)[0, 1]:+.2f\}$ ")
plt.suptitle("Figura 5. Consumo frente a las covariables", y=1.02, fontsize=11)
plt.tight_layout(); plt.show()

print("Correlaciones con el consumo:",
      {v: round(float(np.corrcoef(df[v], y)[0, 1]), 3) for v in ["temperatura_media_c", "clientes_comerciales",
                                                                "dias_habiles"]})
# Razón serie/nivel (estacionalidad + irregular) frente a la temperatura y a los días hábiles
rz = (y / y.rolling(12, center=True).mean()).rolling(2, center=True).mean().shift(-1).dropna()
print("Correlación de la razón serie/nivel con temperatura:", round(np.corrcoef(rz, df.loc[rz.index,
                                             "temperatura_media_c"])[0, 1], 3),
      "| con días hábiles:", round(np.corrcoef(rz, df.loc[rz.index, "dias_habiles"])[0, 1], 3))
# Clientes: salto puntual de abril de 2019
print("Clientes comerciales (mar, abr, may 2019):",
      df.loc["2019-03-01":"2019-05-01", "clientes_comerciales"].tolist())
print("Consumo medio por temperatura:")
print(y.groupby(pd.cut(df["temperatura_media_c"], [0, 12, 14, 16, 18, 25])).mean().round(1).to_string())
```

Correlaciones con el consumo: {'temperatura_media_c': 0.686, 'clientes_comerciales': 0.319, 'dias_habiles': 0.016}
 Correlación de la razón serie/nivel con temperatura: 0.736 | con días hábiles: 0.033
 Clientes comerciales (mar, abr, may 2019): [47561, 48620, 48720]
 Consumo medio por temperatura:
 temperatura_media_c
 (0, 12] 97.1
 (12, 14] 95.0
 (14, 16] 95.3
 (16, 18] 100.1
 (18, 25] 116.8



Descripción (Figura 5). La **temperatura** es la covariable más asociada ($\rho=+0,69$ lineal). Como la relación es una U, el coeficiente lineal subestima la dependencia: la razón serie/nivel alcanza $\rho=+0,74$ y el perfil estacional correlaciona +0,80 con la temperatura mensual. Los **clientes** se asocian débilmente ($\rho=+0,32$): crecen de 42,4 a 54,8 mil empalmes mientras el consumo se estanca en 2018-2019, y dos series con tendencia pueden parecer relacionadas sin estarlo (regresión espuria). En abril de 2019 saltan de 47.561 a 48.620 (+1.059, unas diez veces el aumento mensual habitual), coherente con el centro comercial, pero el consumo **no muestra escalón** (-0,9 % interanual en abril y +2,7 % a +6,7 % de junio a septiembre): los nuevos clientes aportan poco consumo o su efecto se diluye en el ruido. Los **días hábiles** casi no se correlacionan ($\rho=+0,02$).

Decisión — enfoque univariado. Trabajo solo con `consumo_gwh` porque (1) la temperatura explica la estacionalidad pero usarla para pronosticar exige proyectarla, lo que agrega incertidumbre; (2) los clientes crecen con la tendencia y no aportan información nueva; y (3) los días hábiles no muestran señal.

3.6 Delimitación y magnitud del efecto COVID-19

▼ Código

```
# =====
# 3.6 Delimitación y magnitud del efecto COVID-19 (contrafactual simple)
# =====
# Contrafactual: promedio del mismo mes de 2018 y 2019 (nivel anual estable entre ambos años)
base = pd.concat([y["2018"], y["2019"]]).groupby(lambda f: f.month).mean()
contra = pd.Series(base.reindex(y.index.month).values, index=y.index)
brecha = (y / contra - 1) * 100

ventana = slice("2020-01-01", "2022-12-01")
print("Desviación % respecto del contrafactual (2020-2022):")
```

```
print(brecha[ventana].round(1).to_frame("desv. %").T.to_string())

w = slice(COVID_INI, COVID_FIN)
perdida = (contra[w] - y[w]).sum()
print(f"\nEnergía no consumida mar-2020 a dic-2021: {perdida:.1f} GWh "
      f"({100 * perdida / contra[w].sum():.1f}% de lo esperado)")
print(f"Mes de mayor caída: {brecha[w].idxmin():%Y-%m} ({brecha[w].min():.1f}%)")
bajo = brecha[COVID_INI:"2022-12-01"][brecha[COVID_INI:"2022-12-01"] < -3]
print(f"Último mes con desviación < -3%: {bajo.index.max():%Y-%m} -> retorno sostenido desde "
      f"{bajo.index.max() + pd.DateOffset(months=1):%Y-%m}")
```

```
Desviación % respecto del contrafactual (2020-2022):
fecha 2020-01-01 2020-02-01 2020-03-01 2020-04-01 2020-05-01 2020-06-01 2020-07-01 2020-08-01 2020-09-01 2020-10-01
2020-11-01 2020-12-01 2021-01-01 2021-02-01 2021-03-01 2021-04-01 2021-05-01 2021-06-01 2021-07-01 2021-08-01
2021-09-01 2021-10-01 2021-11-01 2021-12-01 2022-01-01 2022-02-01 2022-03-01 2022-04-01 2022-05-01 2022-06-01 2022-07-01
2022-08-01 2022-09-01 2022-10-01 2022-11-01 2022-12-01
desv. % -3.8 -0.4 -8.4 -20.5 -24.4 -19.4 -13.4 -12.5 -8.5 -1.9
-11.2 -11.5 -2.7 -4.6 -6.3 -3.5 -7.0 -4.9 -3.7 -1.1
0.1 -10.4 -8.8 -3.6 -5.9 0.4 -0.8 0.2 2.6 1.5 -0.4
2.4 4.5 -2.6 -0.8 0.5

Energía no consumida mar-2020 a dic-2021: 194.6 GWh (8.4% de lo esperado)
Mes de mayor caída: 2020-05 (-24.4%)
Último mes con desviación < -3%: 2022-01 -> retorno sostenido desde 2022-02
```

Descripción. Para medir el efecto necesito un contrafactual: el promedio del mismo mes de 2018 y 2019, dos años de nivel casi idéntico (107,1 y 107,2 GWh) que evitan extrapolar una tendencia detenida. El efecto **comienza en marzo de 2020** (-8,4 %; enero y febrero, -3,8 % y -0,4 %, son menores), alcanza su **máximo en mayo de 2020** (-24,4 %), se recupera hasta diciembre (-2,7 %) y recae en octubre-noviembre de 2020 (-11,2 % y -11,5 %) y de 2021 (-10,4 % y -8,8 %). Con el criterio de desviación mayor a -3 %, la **normalización sostenida llega en febrero de 2022**, y la ventana total es **marzo de 2020 a enero de 2022**. La energía no consumida entre marzo de 2020 y diciembre de 2021 es de **194,6 GWh** (8,4 % de lo esperado), estimación conservadora porque el contrafactual supone crecimiento nulo, cuando la tendencia histórica rondaba 2,8 % anual.

4. Descomposición de la serie e identificación de componentes

Aplico la **descomposición clásica por medias móviles** (`seasonal_decompose`, periodo 12): estima la tendencia-ciclo con una media móvil centrada 2×12 , promedia por mes la razón (o diferencia) entre la serie y esa tendencia para obtener un componente estacional fijo, y deja el residuo como error. La elegí por su transparencia y su relación directa con la evaluación; sus limitaciones y la alternativa STL (Cleveland et al., 1990) se comentan al final.

Decisión 3 — Esquema aditivo o multiplicativo

En el esquema **aditivo** ($y_t = T_t + S_t + R_t$) la estacionalidad tiene el mismo tamaño en GWh sin importar el nivel; en el **multiplicativo** ($y_t = T_t \times S_t \times R_t$) es un porcentaje del nivel y crece con él.

▼ Código

```
# =====
# 4.1 Descomposición clásica: aditiva vs. multiplicativa
# =====
dec_add = seasonal_decompose(y, model="additive", period=12)
dec_mul = seasonal_decompose(y, model="multiplicative", period=12)

# Criterio de elección: un esquema es adecuado si su residuo es homogéneo respecto del nivel.
# Medimos la dispersión del residuo RELATIVA al nivel (tendencia) en el tramo sin COVID.
tramo = [(pd.Timestamp("2015-07-01"), pd.Timestamp("2019-12-01")),
         (pd.Timestamp("2022-01-01"), pd.Timestamp("2023-06-01"))]

def sin_covid(serie):
    return pd.concat([serie[a:b] for a, b in tramo]).dropna()

r_add = sin_covid(dec_add.resid / dec_add.trend * 100) # % del nivel
r_mul = sin_covid((dec_mul.resid - 1) * 100) # % (ya es razón)
tabla_esq = pd.DataFrame({
    "aditivo": {"desv. residuo (% del nivel)": r_add.std(),
               "corr(|residuo|, nivel)": np.corrcoef(sin_covid(dec_add.resid).abs(), sin_covid(dec_add.trend))[0, 1]},
    "multiplicativo": {"desv. residuo (% del nivel)": r_mul.std(),
                      "corr(|residuo|, nivel)": np.corrcoef(sin_covid(dec_mul.resid - 1).abs(), sin_covid(dec_mul.trend))
                      [0, 1]},
}).round(3)
display(tabla_esq)
```

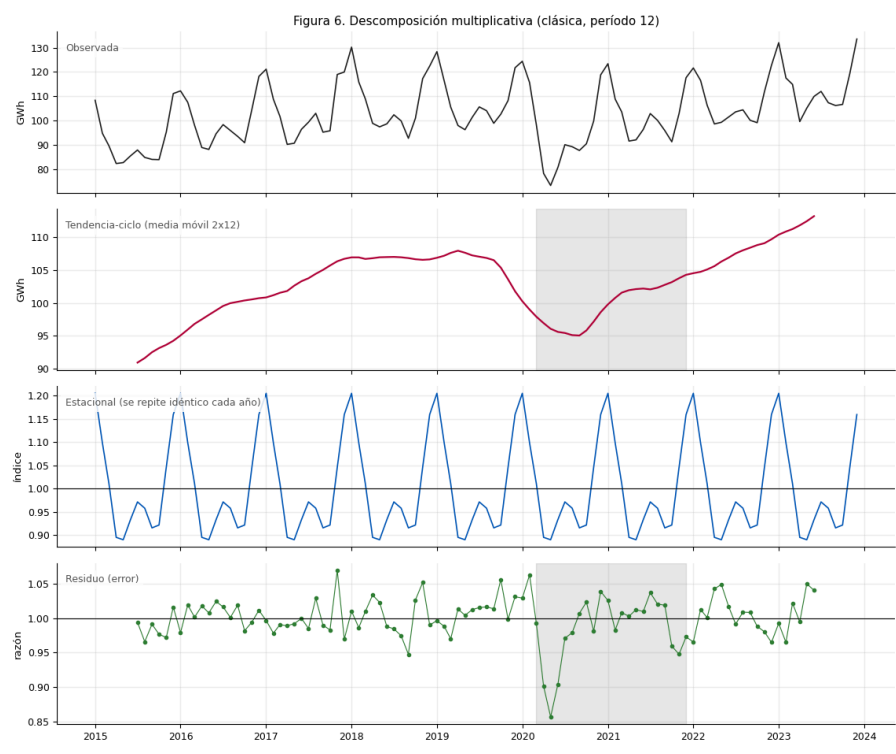
	aditivo	multiplicativo
desv. residuo (% del nivel)	2.358	2.456
corr(residuo , nivel)	0.194	0.185

Elemento	Detalle
Qué decidí	Usar el esquema multiplicativo para analizar los componentes.
A favor	La amplitud relativa es estable (~30 %, de 31,8 % a 30,0 %) mientras la absoluta crece de 28,8 a 34,1 GWh; el índice se lee en porcentaje (enero +20,5 %, mayo -11,0 %); la tendencia (+2,8 % anual) eleva el nivel sobre el que actúa la estacionalidad; y el caso describe una estacionalidad proporcional.
En contra	La elasticidad es 0,31, la desviación móvil no crece con la media y la tabla muestra residuos casi idénticos en ambos esquemas (2,36 % del nivel el aditivo y 2,46 % el multiplicativo; correlación residuo-nivel 0,19 en ambos): los datos no rechazan con fuerza el aditivo .
Alternativa descartada	El aditivo sigue siendo razonable, pero prefiero el multiplicativo por la amplitud relativa estable, su interpretación en porcentaje y su vínculo con el logaritmo que probablemente use al modelar. STL queda para esa etapa.
Consecuencia	Los componentes se leen como porcentaje del nivel. En la Semana 2 verificaré la elección comparando log y Box-Cox; la ambigüedad es una limitación reconocida.

4.1 Componentes estimados

▼ Código

```
# =====  
# 4.2 Componentes del esquema multiplicativo  
# =====  
fig, axes = plt.subplots(4, 1, figsize=(11.5, 9.5), sharex=True)  
axes[0].plot(dec_mul.observed, color=NEGRO, lw=1.2); axes[0].set_ylabel("GWh")  
axes[0].set_title("Figura 6. Descomposición multiplicativa (clásica, período 12)")  
axes[0].text(0.01, 0.88, "Observada", transform=axes[0].transAxes, fontsize=9, color="#555555",  
bbox=dict(facecolor="white", alpha=0.85, edgecolor="none"))  
axes[1].plot(dec_mul.trend, color=ROJO, lw=1.6); axes[1].set_ylabel("GWh")  
axes[1].axvspan(COVID_INI, COVID_FIN, color=GRIS, alpha=0.25)  
axes[1].text(0.01, 0.88, "Tendencia-ciclo (media móvil 2x12)", transform=axes[1].transAxes, fontsize=9, color="#555555",  
bbox=dict(facecolor="white", alpha=0.85, edgecolor="none"))  
axes[2].plot(dec_mul.seasonal, color=AZUL, lw=1.2); axes[2].axhline(1, color="k", lw=0.8)  
axes[2].set_ylabel("índice")  
axes[2].text(0.01, 0.88, "Estacional (se repite idéntico cada año)", transform=axes[2].transAxes, fontsize=9,  
color="#555555", bbox=dict(facecolor="white", alpha=0.85, edgecolor="none"))  
axes[3].plot(dec_mul.resid, "o-", color=VERDE, lw=0.8, ms=3); axes[3].axhline(1, color="k", lw=0.8)  
axes[3].axvspan(COVID_INI, COVID_FIN, color=GRIS, alpha=0.25); axes[3].set_ylabel("razón")  
axes[3].text(0.01, 0.88, "Residuo (error)", transform=axes[3].transAxes, fontsize=9, color="#555555",  
bbox=dict(facecolor="white", alpha=0.85, edgecolor="none"))  
plt.tight_layout(); plt.show()
```



▼ Código

```
# =====  
# 4.3 Cuantificación de componentes
```

```
# =====
T, S, R = dec_mul.trend.dropna(), dec_mul.seasonal, dec_mul.resid.dropna()

# Índice estacional (idéntico cada año por construcción)
idx_est = S.groupby(S.index.month).first()
print("Índice estacional mensual:", dict(zip(idx_est.index, idx_est.round(3))))
print(f"Mes máximo: {idx_est.idxmax()} ({idx_est.max():.3f}) · mes mínimo: {idx_est.idxmin()} ({idx_est.min():.3f}) "
      f"· razón máx/mín = {idx_est.max() / idx_est.min():.2f}")

# Fuerza de tendencia y estacionalidad (Hyndman y Athanasopoulos, 2021), sobre logaritmos
# porque el esquema es multiplicativo: log(y) = log(T) + log(S) + log(R)
lT, lS, lR = np.log(T), np.log(S.loc[R.index]), np.log(R)
lT = lT.loc[R.index]
Ft = max(0, 1 - lR.var() / (lT + lR).var())
Fs = max(0, 1 - lR.var() / (lS + lR).var())
print(f"Fuerza de la tendencia      Ft = {Ft:.2f}")
print(f"Fuerza de la estacionalidad Fs = {Fs:.2f}")

# Tendencia: crecimiento por tramos
# (los extremos de la media móvil están contaminados cerca del shock: se eligen tramos prudentes)
for a, b in [ ("2015-07-01", "2018-01-01"), ("2018-01-01", "2019-06-01"),
              ("2019-06-01", "2020-09-01"), ("2020-09-01", "2022-06-01"), ("2022-06-01", "2023-06-01") ]:
    print(f"Tendencia {a[:7]} -> {b[:7]}: {T[a:].1f} -> {T[b:].1f} GWh ({100 * (T[b] / T[a] - 1):+.1f}%)")
crec_anual = 100 * ((T["2023-06-01"] / T["2015-07-01"]) ** (12 / ((T.index.get_loc(pd.Timestamp("2023-06-01"))
- T.index.get_loc(pd.Timestamp("2015-07-01"))))) - 1)
print(f"Crecimiento medio anual de la tendencia (2015-07 a 2023-06): {crec_anual:.2f}%")

# Ciclo: movimientos de baja frecuencia de la tendencia respecto de su recta de largo plazo
t = np.arange(len(T)); b1, b0 = np.polyfit(t, np.log(T.values), 1)
ciclo = pd.Series(np.log(T.values) - (b0 + b1 * t), index=T.index) * 100
print(f"\nCiclo (desv. % de la tendencia respecto de su recta log-lineal): mín {ciclo.min():.1f}% "
      f"f({ciclo.idxmin():%Y-%m}), máx {ciclo.max():.1f}% ({ciclo.idxmax():%Y-%m})")

# Error
print(f"\nResiduo: media {R.mean():.3f}, desv. estándar {R.std():.3f} (≈{100 * R.std():.1f}% del nivel)")
print("Mayores desvíos del residuo (razón):")
print(R.sub(1).abs().sort_values(ascending=False).head(8).round(3).to_string())
```

Índice estacional mensual: {1: 1.205, 2: 1.098, 3: 1.01, 4: 0.895, 5: 0.89, 6: 0.933, 7: 0.971, 8: 0.958, 9: 0.915, 10: 0.921, 11: 1.046, 12: 1.159}

Mes máximo: 1 (1.205) · mes mínimo: 5 (0.890) · razón máx/mín = 1.35

Fuerza de la tendencia Ft = 0.75

Fuerza de la estacionalidad Fs = 0.90

Tendencia 2015-07 -> 2018-01: 90.9 -> 106.9 GWh (+17.6%)

Tendencia 2018-01 -> 2019-06: 106.9 -> 107.2 GWh (+0.3%)

Tendencia 2019-06 -> 2020-09: 107.2 -> 95.1 GWh (-11.4%)

Tendencia 2020-09 -> 2022-06: 95.1 -> 106.9 GWh (+12.5%)

Tendencia 2022-06 -> 2023-06: 106.9 -> 113.2 GWh (+5.9%)

Crecimiento medio anual de la tendencia (2015-07 a 2023-06): 2.81%

Ciclo (desv. % de la tendencia respecto de su recta log-lineal): mín -9.3% (2020-09), máx 5.6% (2018-01)

Residuo: media 1.000, desv. estándar 0.033 (≈3.3% del nivel)

Mayores desvíos del residuo (razón):

fecha

2020-05-01 0.144

2020-04-01 0.099

2020-06-01 0.096

2017-11-01 0.070

2020-02-01 0.063

2019-10-01 0.057

2018-09-01 0.052

2018-11-01 0.052

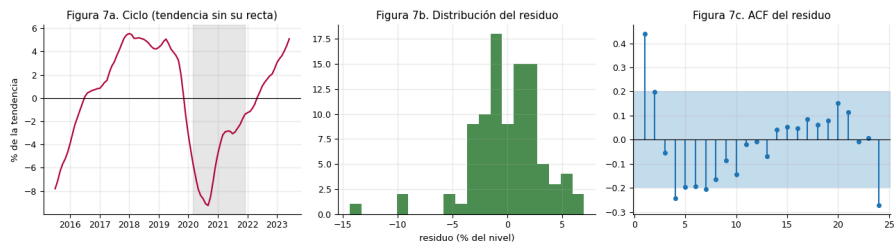
▼ Código

```
# =====
# 4.4 Ciclo y error: visualización
# =====
fig, axes = plt.subplots(1, 3, figsize=(13, 3.6))
axes[0].plot(ciclo, color=R0J0); axes[0].axhline(0, color="k", lw=0.8)
axes[0].axvspan(COVID_INI, COVID_FIN, color=GRIS, alpha=0.25)
axes[0].set_ylabel("% de la tendencia"); axes[0].set_title("Figura 7a. Ciclo (tendencia sin su recta)")
axes[1].hist(R.sub(1) * 100, bins=20, color=VERDE, alpha=0.85)
axes[1].set_xlabel("residuo (% del nivel)"); axes[1].set_title("Figura 7b. Distribución del residuo")
plot_acf(R.values, lags=24, ax=axes[2], zero=False, title="Figura 7c. ACF del residuo")
plt.tight_layout(); plt.show()

# Dispersión del residuo con y sin la ventana COVID
cov = (R.index >= COVID_INI) & (R.index <= COVID_FIN)
print(f"Desv. del residuo: con COVID {R.std():.4f} · sin COVID {R[~cov].std():.4f} · solo COVID {R[cov].std():.4f}")
n_r = len(R); banda_r = 1.96 / np.sqrt(n_r)
ar = acf(R.values, nlags=24)
print(f"ACF del residuo: rezago 1 = {ar[1]:+.2f}, rezago 2 = {ar[2]:+.2f}, rezago 12 = {ar[12]:+.2f} "
      f"f(banda ±{banda_r:.2f})")
```

Desv. del residuo: con COVID 0.0325 · sin COVID 0.0254 · solo COVID 0.0472

ACF del residuo: rezago 1 = +0.44, rezago 2 = +0.20, rezago 12 = -0.01 (banda ±0.20)



Tendencia. La tendencia-ciclo (Figura 6) sube de 90,9 GWh (julio de 2015) a 106,9 (enero de 2018), +17,6 %; se estanca hasta junio de 2019 (+0,3 %); cae -11,4 % hasta septiembre de 2020 (95,1 GWh); se recupera +12,5 % hasta junio de 2022 (106,9) y crece +5,9 % hasta junio de 2023 (113,2). El crecimiento medio anual es 2,8 %. La media móvil suaviza los quiebres (el descenso de 2020 aparece más gradual de lo real) y se pierden seis observaciones por extremo.

Estacionalidad. Es anual ($s=12$), idéntica cada año por construcción y muy intensa: el índice va de 0,890 (mayo) a 1,205 (enero), una razón máx/mín de 1,35. Su fuerza, $F_s = 1 - \text{Var}(R)/\text{Var}(S+R)$ sobre logaritmos (Hyndman y Athanasopoulos, 2021), es **0,90**, frente a **0,75** de la tendencia (F_t): la estacionalidad domina.

Ciclo. Un ciclo es una fluctuación de largo plazo sin periodo fijo. En estos 9 años no hay uno recurrente: solo el **estancamiento de 2018-2019** (+5,6 % sobre la recta de largo plazo en enero de 2018) y el **hundimiento de 2020** (-9,3 % en septiembre, Figura 7a), que es un **shock exógeno** y no un ciclo económico (de 5 a 10 años, imposible de identificar con 9). La estructura es *tendencia + estacionalidad + intervención*, con ciclo no identificable.

Error. El residuo tiene media 1,000 y desviación 0,033 (3,3 % del nivel; 2,5 % sin COVID y 4,7 % dentro). Los mayores desvíos son abril, mayo y junio de 2020 (-9,9 %, -14,4 % y -9,6 %), noviembre de 2017 (+7,0 %) y febrero de 2020 (+6,3 %). **No es ruido blanco:** $\rho_1=+0,44$ y $\rho_2=+0,20$ superan la banda de $\pm 0,20$, y el histograma tiene cola negativa por el shock.

Lectura de las Figuras 7a-7c. El ciclo estimado muestra la meseta previa al COVID y la depresión posterior; el residuo se concentra en torno a 0 con cola izquierda (meses COVID). Su autocorrelación positiva indica que la descomposición no absorbió el efecto COVID, que dura meses y no cabe en una estacionalidad fija ni en una media móvil suave.

Limitaciones. La descomposición clásica supone estacionalidad constante, no es robusta a atípicos ni quiebres y pierde datos en los extremos. STL permite estacionalidad cambiante y ajuste robusto; la consideraré al modelar.

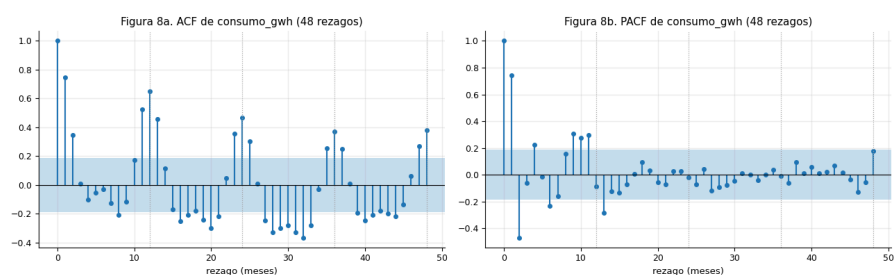
5. Análisis de autocorrelación (ACF y PACF)

La **ACF** mide la correlación entre y_t e y_{t-k} , incluidos efectos indirectos; la **PACF** la mide **descontando** los rezagos $1, \dots, k-1$ (Box et al., 2015). Las calculo hasta el rezago 48 (cuatro ciclos anuales) sobre la serie en nivel, con $n=108$. Una barra es significativa si supera la banda de 95 % de $\pm 1,96/\sqrt{n} = \pm 0,189$ (los gráficos de `statsmodels` usan la fórmula de Bartlett, a veces algo más ancha).

5.1 Correlogramas de la serie

▼ Código

```
# =====
# 5.1 ACF y PACF de la serie (nivel), 48 rezagos
# =====
NLAGS = 48
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
plot_acf(y.values, lags=NLAGS, ax=axes[0], title="Figura 8a. ACF de consumo_gwh (48 rezagos)")
plot_pacf(y.values, lags=NLAGS, ax=axes[1], method="yw", title="Figura 8b. PACF de consumo_gwh (48 rezagos)")
for ax in axes:
    ax.set_xlabel("rezago (meses)")
    for k in (12, 24, 36, 48):
        ax.axvline(k, color=GRIS, ls=":", lw=0.8)
plt.tight_layout(); plt.show()
```



▼ Código

```
# =====
# 5.2 Valores y rezagos significativos (banda ±1,96/√n)
# =====
n = len(y); banda = 1.96 / np.sqrt(n)
r = acf(y.values, nlags=NLAYS)
p = pacf(y.values, nlags=NLAYS, method="ywmm")

clave = [1, 2, 3, 6, 9, 12, 13, 18, 24, 36, 48]
tabla = pd.DataFrame({"ACF": r[clave], "PACF": p[clave]}, index=pd.Index(clave, name="rezago")).round(3)
tabla["ACF signif."] = np.abs(r[clave]) > banda
tabla["PACF signif."] = np.abs(p[clave]) > banda
display(tabla)

print(f"n = {n}; banda de confianza 95% = ±{banda:.3f}")
print("Rezagos ACF significativos :", [k for k in range(1, NLAYS + 1) if abs(r[k]) > banda])
print("Rezagos PACF significativos:", [k for k in range(1, NLAYS + 1) if abs(p[k]) > banda])
print("ACF en múltiplos de 12      :", {k: round(float(r[k]), 3) for k in (12, 24, 36, 48)})

n = 108; banda de confianza 95% = ±0.189
Rezagos ACF significativos : [1, 2, 8, 11, 12, 13, 16, 17, 19, 20, 21, 23, 24, 25, 27, 28, 29, 30, 31, 32, 33, 35, 36, 37,
39, 40, 41, 43, 44, 47, 48]
Rezagos PACF significativos: [1, 2, 4, 6, 9, 10, 11, 13]
ACF en múltiplos de 12      : {12: 0.652, 24: 0.468, 36: 0.369, 48: 0.378}
```

	ACF	PACF	ACF signif.	PACF signif.
rezago				
1	0.745	0.745	True	True
2	0.345	-0.469	True	True
3	0.007	-0.061	False	False
6	-0.032	-0.235	False	True
9	-0.115	0.307	False	True
12	0.652	-0.087	True	False
13	0.459	-0.283	True	True
18	-0.179	0.095	False	False
24	0.468	-0.022	True	False
36	0.369	-0.011	True	False
48	0.378	0.178	True	False

5.2 Interpretación

Persistencia. La ACF parte en $\rho_1 = 0,745$ y $\rho_2 = 0,345$, significativos, y cae a casi cero en el rezago 3 ($\rho_3 = 0,007$): meses consecutivos se parecen por la suavidad del ciclo estacional y la persistencia del nivel. En la serie desestacionalizada ρ_1 sube a 0,87 (Figura 9a), lo que confirma una **fuerte persistencia del nivel** ligada a la tendencia y los cambios de nivel: la media no es constante y la serie **no es estacionaria**.

Estacionalidad. Los rezagos 12, 24, 36 y 48 son significativos ($\rho_{12}=0,652$; $\rho_{24}=0,468$; $\rho_{36}=0,369$; $\rho_{48}=0,378$), igual que los vecinos 11 y 13 ($\rho_{13}=0,459$). **Picos en múltiplos de 12 con decaimiento lento** son la firma de una estacionalidad anual estable con no estacionariedad; en una serie estacionaria se atenuarían mucho más rápido. Entre picos la ACF oscila y es negativa en los semiciclos (rezagos 16-21, 27-33 y 39-44, con mínimo de -0,37 en el 32), pero en el rezago 6 es casi cero ($\rho_6=-0,03$) porque el perfil **no es una onda simple**: tiene un máximo estival dominante y un repunte invernal menor. Que 31 de 48 rezagos sean significativos no implica 31 dependencias reales: una serie no estacionaria arrastra correlación espuria.

Dependencia directa (PACF). Tiene dos valores grandes al inicio ($\phi_{11}=0,745$ y $\phi_{22}=-0,469$) y valores dispersos en los rezagos 4, 6, 9, 10, 11 y 13, sin pico en el 12 (-0,087), 24 ni 36 (-0,022 y -0,011). Un positivo seguido de un negativo es compatible con dependencia autorregresiva de orden dos, pero la lectura es **provisoria**: en una serie no estacionaria la PACF mezcla dependencia real con tendencia y estacionalidad.

Límite. El correlograma permite **diagnosticar** (no estacionariedad, estacionalidad anual) pero **no identificar** los órdenes $(p,q)(P,Q)$, lo que corresponde después de estacionarizar.

5.3 Contraste: serie desestacionalizada y residuo

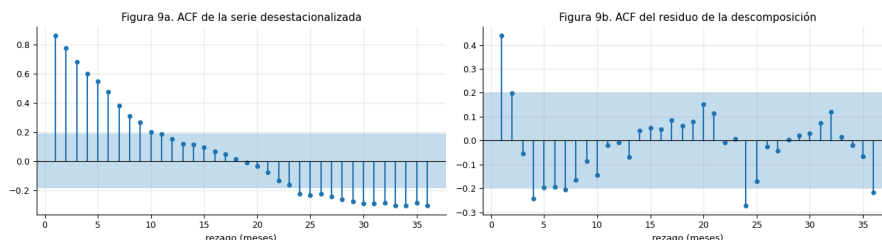
▼ Código

```
# =====
# 5.3 Contraste: ¿qué ocurre con la ACF al retirar tendencia y estacionalidad?
# =====
# Serie desestacionalizada (y / índice estacional) y residuo de la descomposición
y_dsz = (y / dec_mul.seasonal).dropna()
fig, axes = plt.subplots(1, 2, figsize=(13, 3.6))
plot_acf(y_dsz.values, lags=36, ax=axes[0], zero=False, title="Figura 9a. ACF de la serie desestacionalizada")
plot_acf(R.values, lags=36, ax=axes[1], zero=False, title="Figura 9b. ACF del residuo de la descomposición")
```

```
for ax in axes: ax.set_xlabel("rezago (meses)")
plt.tight_layout(); plt.show()

rd = acf(y_dsz.values, nlags=36)
print("ACF desestacionalizada: rezago 1 =", round(rd[1], 2), "| rezago 12 =", round(rd[12], 2), "| rezago 24 =",
      round(rd[24], 2))
ra = acf(R.values, nlags=36)
print("ACF del residuo : rezago 1 =", round(ra[1], 2), "| rezago 12 =", round(ra[12], 2), "| rezago 24 =",
      round(ra[24], 2))
```

ACF desestacionalizada: rezago 1 = 0.87 | rezago 12 = 0.15 | rezago 24 = -0.23
 ACF del residuo : rezago 1 = 0.44 | rezago 12 = -0.01 | rezago 24 = -0.27



Lectura de las Figuras 9a y 9b. Al dividir por el índice estacional desaparecen los picos en 12, 24 y 36 ($\rho_{12}=0,15$ y $\rho_{24}=-0,23$): esas correlaciones venían de la estacionalidad. Queda una autocorrelación de orden 1 muy alta ($\rho_1=0,87$) y lenta, atribuible a la tendencia y al COVID. En el residuo baja a $\rho_1=+0,44$ y $\rho_{12}=-0,01$: la estacionalidad quedó bien capturada, pero resta dependencia de corto plazo por el COVID y los cambios de nivel.

6. Interpretación técnica integrada

6.1 Convergencia de la evidencia

Componente	Gráficos exploratorios	Descomposición	ACF / PACF
Tendencia	Crece 2015-2018, se estanca 2018-2019, cae en 2020 y se recupera (Figuras 1 y 4a)	De 90,9 a 113,2 GWh, +2,8 % anual; $F_t=0,75$	Decaimiento lento; $\rho_1=0,87$ desestacionalizada: media no constante
Estacionalidad	Máximo en enero (1,19), mínimo en mayo (0,89), repunte invernal; explicada por temperatura (Figuras 3a-3c)	Índice 0,890-1,205; $F_s=0,90$, dominante	Picos en 12, 24, 36 y 48 (0,65; 0,47; 0,37; 0,38) con decaimiento lento
Ciclo	Solo estancamiento 2018-2019 y hundimiento de 2020	Desvío de +5,6 % a -9,3 % sobre la recta: sin periodicidad	Sin señal distinta de 12
Shocks	Quiebre de marzo de 2020 (mín. -24,4 %; 194,6 GWh perdidos); atípico de 2017; sin escalón en 2019	Residuos extremos en 2020 y 2017-11	Residuo con $\rho_1=+0,44$: queda estructura
Error	Variabilidad de ~10 GWh	Desviación del residuo 3,3 % (2,5 % sin COVID)	Desaparece el pico estacional ($\rho_{12}=-0,01$)

6.2 Conclusión técnica

La serie es **no estacionaria en media, con estacionalidad anual intensa y estable, tendencia creciente con estancamiento en 2018-2019, un shock externo (COVID) y un atípico de medición confirmado**. Tres líneas de evidencia coinciden: los gráficos (perfil repetido, media móvil no constante), la descomposición ($F_s=0,90$, $F_t=0,75$, residuo dominado por 2020) y la ACF (picos en múltiplos de 12 con decaimiento lento, $\rho_1=0,87$ desestacionalizada). La PACF no contradice el diagnóstico, pero por sí sola no identifica órdenes. La estacionalidad se asocia al calor estival y, menos, al frío invernal.

6.3 Implicancias para el modelamiento

1. **Transformación.** Partir con el logaritmo y compararlo con Box-Cox y la serie original en la Semana 2.

2. **Diferenciación.** Los picos de ACF en múltiplos de 12 sugieren $D=1$ y la persistencia del nivel $d=1$; se confirmarán con ADF, KPSS y el criterio de varianza para no sobrediferenciar. La diferencia estacional se calcula con `serie.diff(12)`, no con `np.diff(x, n=12)`, que aplica doce diferencias regulares.
3. **Quiebre COVID.** La diferenciación estacional distorsiona los valores que cruzan el shock: unas 24 de 96 observaciones (25 %). Habrá que elegir entre indicadoras (consumen grados de libertad y suponen una forma funcional) o acotar la muestra (menos de tres ciclos completos).
4. **Atípico de 2017.** Tratarlo como faltante o con una indicadora.
5. **Covariables.** Mantener el enfoque univariado (SARIMA); la temperatura podría entrar en un SARIMAX a costa de proyectarla.
6. **Referencia.** Con $F_5=0,90$, el ingenuo estacional será una vara exigente que el modelo deberá superar.

6.4 Limitaciones

Nueve años de datos (dos afectados por el shock) limitan la precisión estacional y impiden identificar ciclos. El COVID es un único episodio y su magnitud depende de un supuesto contrafactual. La descomposición clásica impone estacionalidad fija y no separa la tendencia del shock. La elección aditivo/multiplicativo tiene evidencia mixta. Los datos son sintéticos: lo dicho sobre el mecanismo físico son hipótesis, no hallazgos empíricos.

7. Conclusiones

1. La serie combina tendencia, estacionalidad anual dominante, un shock por COVID y un atípico de medición; no presenta ciclo identificable.
2. La estacionalidad es la estructura más fuerte ($F_5=0,90$): máximo en enero, mínimo en mayo y repunte en julio, explicados por el calor estival y el frío invernal.
3. La tendencia crece cerca de 2,8 % anual, con estancamiento en 2018-2019 y caída en 2020; el COVID implicó unos 195 GWh no consumidos (8,4 % de lo esperado entre marzo de 2020 y diciembre de 2021).
4. La ACF, con picos en 12, 24, 36 y 48 y decaimiento lento, confirma una serie no estacionaria y estacional; la PACF sugiere dependencia de corto plazo, pero los órdenes se identificarán tras estacionarizar.
5. Para modelar: logaritmo como punto de partida, diferenciación estacional y probablemente regular, y tratamiento explícito del COVID y del atípico de 2017.

8. Referencias

- Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time series analysis: Forecasting and control* (5.^a ed.). Wiley.
- Cleveland, R. B., Cleveland, W. S., McRae, J. E., & Terpenning, I. (1990). STL: A seasonal-trend decomposition procedure based on loess. *Journal of Official Statistics*, 6(1), 3-73.
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3.^a ed.). OTexts. <https://otexts.com/fpp3/>
- Seabold, S., & Perktold, J. (2010). statsmodels: Econometric and statistical modeling with Python. *Proceedings of the 9th Python in Science Conference*, 92-96.
- Tashman, L. J. (2000). Out-of-sample tests of forecasting accuracy: An analysis and review. *International Journal of Forecasting*, 16(4), 437-450.
- Universidad Andrés Bello. (s. f.). *MCDI505 — Análisis de series de tiempo: Caso transversal del curso* [Material de curso]. UNAB Online.

9. Declaración de uso de herramientas de IA

Se utilizó un asistente de IA generativa (Claude, de Anthropic) para elaborar el borrador del código, los análisis y la redacción de este informe. El autor revisó el notebook, verificó que se ejecuta de principio a fin y es responsable de su contenido y de las decisiones metodológicas expuestas.